

L5 总结

L5 总结:记忆与状态(阶段二第一课)

承接 L4 capstone:你写的累积 messages 就是“短期记忆”。本节划清记忆边界,并亲手实现最朴素的长期记忆(存文件 + 下次注入)。本节最大收获其实是两次翻车。

一、核心概念

1. 你早就写过记忆了

capstone 里 `main` 维护的贯穿会话 `messages` = **短期记忆 / 工作记忆**。“把它退了”认得“它”就是它的功劳。也回扣 L2 学习者自己追问出的:“LLM 无持续连接、追问会忘、messages 越滚越长”。

2. 短期记忆活不过重启

`messages` 是内存里的 Python 列表,进程一死随内存释放。查完订单关掉程序再开,agent 就失忆。

3. 三个必须分清的概念

概念	是什么	capstone 例子	活多久
短期记忆	当前对话的 messages	累积的对话历史	一次会话/进程
长期记忆	跨会话记住,存在 messages 之外	用户名/偏好/历史退款	永久(落盘/入库)
状态 state	程序自己维护的事实,不一定给模型看	<code>REFUNDED</code> 、 <code>POINTS</code>	看存哪

记忆 vs 状态:记忆是“塞进 messages 给模型读的”;状态是“代码逻辑自己用的”。`REFUNDED` 是纯状态——防重复退款靠 `if order_id in REFUNDED`,根本没进模型的眼睛。 > **一句话:记忆给模型看,状态给代码用。**

4. 长期记忆 = “存 + 取” 读写闭环

短期记忆是 messages 自动累积、白送的;长期记忆得自己实现两个动作: - **存(write)**:识别到值得记住的事实时(会话中或结束时),写到持久层(文件/DB/向量库)。 - **取(read)**:新会话启动时读出来,注入 **system prompt**。

会话1: "我叫小明,只收顺丰" → remember 工具 → 写进 memory.json
↓ (进程重启)
会话2: 启动时 load → 拼进 system prompt → agent 张口就认得你

心智模型:模型永远无状态(L2 就懂)。所谓“记得”,全是每次开会话把该记的重新喂回它眼前。没有魔法,就是“存下来 + 下次读出来拼进 prompt”。

5. 为什么注入 system 而不是伪装成 user 消息

模型被训练成把 system 当作贯穿整段对话的设定/背景,全程有效;伪装成 user 消息则容易被理解成“用户刚刚说了这句”,多轮里串味。位置要对,角色更要对。

二、作业:给 agent 加长期记忆

homework/agent_with_memory.py,三处新肌肉: - load_memory() / save_memory():读写 memory.json(文件不存在时返回 {} 不崩) - build_system_prompt():把记忆拼进 system prompt(“取”) - remember 工具 + schema:让模型自己决定何时存(“存”;呼应 L4 “把意图折进 tool use”)

跑通标准(分两次运行验证跨重启):

第1次: 你> 我叫小明,以后只收顺丰快递
[工具调用] remember({'key': 'name', 'value': '小明'})
[工具调用] remember({'key': 'preferred_carrier', 'value': '顺丰'})
第2次(重开): 你> 你还记得我吗 → "您是小明,偏好顺丰" 

三、本节两次翻车(比代码值钱)

翻车 1:漏写兜底 return → 静默返回 None

`build_system_prompt` 只写了 `if memory:` 那一支的 `return`,忘了“**memory 为空**”时返回。第一次运行 `memory.json` 还不存在 → `load_memory()` 返回 `{}` → 条件为假 → 函数走到底 → **Python 静默返回 None** → `{"role":"system","content":None}` → API 报 `content should be a string or a list` (炸点离病根 30 行远)。

教训(惯犯!L3 已犯过一次):声明了返回值的函数,必须在每条路径上都 `return`。漏一支 Python 不报错不警告,默默给你 `None`,然后在很远的地方以奇怪方式炸掉。 **自查动作:**写完带返回值的函数,回头扫一眼“每个分支都 `return` 了吗”。

翻车 2(本节最重要):模型“嘴上说记住了”,其实没调工具

第一版跑完,模型回“我已经记下了您的姓名和快递偏好 😊”——但一查 `memory.json` **根本不存在**, `remember` 从头到尾没被调用过。模型只是生成了一句像模像样的话术,并没有执行动作。

- **怎么发现的:**不信话术,去看物证(查 `memory.json`);再加 `print(f"[工具调用] {name}({args})")` 把真实 `tool_calls` 打出来,眼见为实。
- **怎么治本:**prompt 太软(“在必要时...记住偏好”)。改成强命令,并**直接点名禁掉偷懒话术**: > “当用户透露姓名/联系方式/快递偏好等值得长期记住的信息时,你【必须】先调用 `remember` > 工具逐条保存,再回复用户。绝对不要只嘴上说‘已记住’却不调用工具。”

改完模型乖乖调了两次 `remember`(姓名、承运商各一条)。

两条硬道理: 1. 模型的“话”和模型的“动作”是两回事。同一句“我记住了”,可能是真的、也可能是演的,唯一区别在有没有 `tool_calls`。验收 agent 永远看动作,不看话术。(→ L11 可观测性)
2. 模型不肯乖乖调工具时,先怀疑 prompt 太软——把期望写成强命令,并点名禁掉它的偷懒路径。

四、环境问题:Windows 上永久设置 API Key

本节还踩了环境变量的坑(老短板)。Windows 永久设法:

```
[Environment]::SetEnvironmentVariable("DEEPSEEK_API_KEY", "sk-真key", "User")
```

关键坑:VSCode 在启动那一刻就把环境变量拍了快照,改注册表后开新终端标签也读不到,必须完全重启 VSCode。临时救急可在当前窗口:

```
$env:DEEPSEEK_API_KEY =  
[Environment]::GetEnvironmentVariable("DEEPSEEK_API_KEY", "User")
```

另一条路(绑到 conda 环境,更聚焦): `conda env config vars set DEEPSEEK_API_KEY= ...`

五、Quiz 回顾(5/5 全过)

- Q1 messages = 短期记忆,内存列表,进程结束即释放。✅
- Q2 REFUNDED 是状态——给代码用、没进 messages。✅
- Q3 无状态与“记得”不矛盾,靠“存+取”每次重新喂回。✅(还主动点出“对话中也可以存”)
- Q4 注入 system prompt。✅(位置答对,角色由讲解补精确)
- Q5 存不下→L6 上下文压缩;取哪几条→L7 RAG 检索。✅

六、埋点(后续回扣)

- **L6 上下文管理:**messages 越滚越长、记忆存不下 → 截断/摘要/压缩。
- **L7 RAG:**记忆多了怎么“按相关性检索该取哪几条”。
- **L10 错误处理:**模型不执行动作、工具调用失败的兜底与重试。
- **L11 可观测性:**[工具调用] 那行 print 就是日志/tracing 的雏形,届时正式做。